

AI-Powered GitHub Code Review Assistant

An Intelligent Developer Tool for Automated
Pull Request Analysis and Code Review

Full Project Specification

Project Type:	Software Engineering / AI Integration
Frontend:	Next.js, TypeScript, Tailwind CSS
Backend:	Python, FastAPI
Database:	PostgreSQL
AI:	Google Gemini API
Integration:	GitHub REST API and Webhooks
Development Environment:	Local Development – No Docker Required

Kisho Jeyapragash

2026

Contents

1	Project Overview	4
2	Problem Statement	5
3	Project Goal	5
4	Objectives	5
5	Target Users	6
6	Core System Workflow	6
7	Technology Stack	7
8	Development Constraints	8
9	Major System Modules	9
9.1	GitHub Integration Module	9
9.2	Webhook Processing Module	9
9.3	Pull Request Analysis Module	10
9.4	Static Analysis Module	10
9.5	AI Review Engine	10
10	AI Review Categories	11
11	Severity Classification	11
11.1	High Severity	11
11.2	Medium Severity	11
11.3	Low Severity	12
12	Structured AI Output	12
13	Example Review Scenario	12
14	False Positive Reduction	13
15	Human-in-the-Loop Design	13
16	Database Design	14
16.1	Users Table	14
16.2	Repositories Table	15
16.3	Pull Requests Table	15
16.4	Reviews Table	15
16.5	Review Findings Table	15

16.6 Developer Feedback Table	16
17 Backend API Design	16
17.1 GitHub Webhook	16
17.2 Repositories	16
17.3 Pull Requests	16
17.4 Reviews	17
17.5 Findings	17
17.6 Feedback	17
18 Frontend Dashboard	17
18.1 Dashboard Home	17
18.2 Repositories Page	17
18.3 Pull Requests Page	18
18.4 Review Details	18
19 GitHub Review Output	18
20 Project-Specific RAG	19
21 Security Requirements	20
22 Recommended Project Structure	21
23 Minimum Viable Product	22
24 Development Phases	22
24.1 Phase 1 – Foundation	22
24.2 Phase 2 – GitHub Integration	23
24.3 Phase 3 – Static Analysis	23
24.4 Phase 4 – Gemini Integration	24
24.5 Phase 5 – Review Engine	24
24.6 Phase 6 – GitHub Feedback	24
24.7 Phase 7 – Dashboard	25
24.8 Phase 8 – Project Context / RAG	25
24.9 Phase 9 – Evaluation	25
25 Testing Strategy	26
25.1 Unit Testing	26
25.2 Integration Testing	26
25.3 End-to-End Testing	26
26 Evaluation Metrics	27
26.1 Detection Rate	27
26.2 Precision	27

26.3 Recall	27
26.4 False Positive Rate	27
26.5 Severity Accuracy	27
26.6 Review Latency	27
27 Evaluation Dataset	28
28 Cost Strategy	28
29 Possible Future Improvements	29
30 Features That Should Not Be Added Initially	30
31 Portfolio Value	30
31.1 Software Engineering	31
31.2 Backend Engineering	31
31.3 Frontend Engineering	31
31.4 AI Engineering	31
31.5 Security	32
32 Potential Interview Discussion	32
33 Success Criteria	33
34 Final System Vision	33
35 Recommended Development Principle	34
36 Conclusion	35

1 Project Overview

The **AI-Powered GitHub Code Review Assistant** is an intelligent developer tool designed to automatically analyse code changes submitted through GitHub Pull Requests.

When a developer creates or updates a Pull Request, the system receives the event through a GitHub Webhook, retrieves the changed files and code diffs, performs deterministic static analysis, and then uses the Google Gemini API to perform contextual code analysis.

The system attempts to identify issues such as:

- Security vulnerabilities
- Potential software bugs
- Missing input validation
- Poor error handling
- Performance problems
- Database-related issues
- Maintainability concerns
- Code quality problems
- Violations of project-specific development standards

Instead of simply reporting that a piece of code is “bad”, the system should explain:

1. What the potential problem is
2. Why the problem matters
3. Where the problem occurs
4. How serious the problem may be
5. How the developer could improve the code

The resulting review can then be posted directly to the GitHub Pull Request and stored for viewing through a web dashboard.

2 Problem Statement

Code review is an important part of professional software development. However, manual code review can require significant developer time, especially when teams handle many Pull Requests.

Reviewers may also overlook problems because of:

- Large code changes
- Limited review time
- Repetitive review tasks
- Different developer experience levels
- Inconsistent coding standards
- Human error

Traditional static-analysis tools can detect predefined patterns, but they may not understand the wider context or intention behind a code change.

Large Language Models can analyse code semantically, but relying only on an LLM may create unreliable findings or false positives.

Therefore, this project proposes a **hybrid code-review system** combining:

Static Analysis + Project Context + AI Analysis + Human Review

The goal is not to replace human code reviewers. Instead, the system acts as an intelligent first-level reviewer that helps developers and reviewers identify potential problems earlier.

3 Project Goal

The main goal is to design and implement an intelligent GitHub-integrated code-review platform capable of analysing Pull Request changes and providing structured, understandable and actionable feedback to developers.

4 Objectives

The main objectives are:

1. Integrate the system with GitHub repositories.
2. Detect Pull Request creation and update events using GitHub Webhooks.
3. Retrieve changed files and code differences through the GitHub REST API.

4. Perform deterministic static analysis before AI analysis.
5. Analyse relevant code changes using the Gemini API.
6. Generate structured code-review findings.
7. Categorise detected issues according to severity.
8. Provide explanations and recommended improvements.
9. Publish useful review feedback to GitHub Pull Requests.
10. Store review history in PostgreSQL.
11. Develop a dashboard for monitoring repositories, Pull Requests and review results.
12. Reduce unnecessary AI-generated comments and false positives.
13. Later introduce project-specific coding standards using retrieval-based context.

5 Target Users

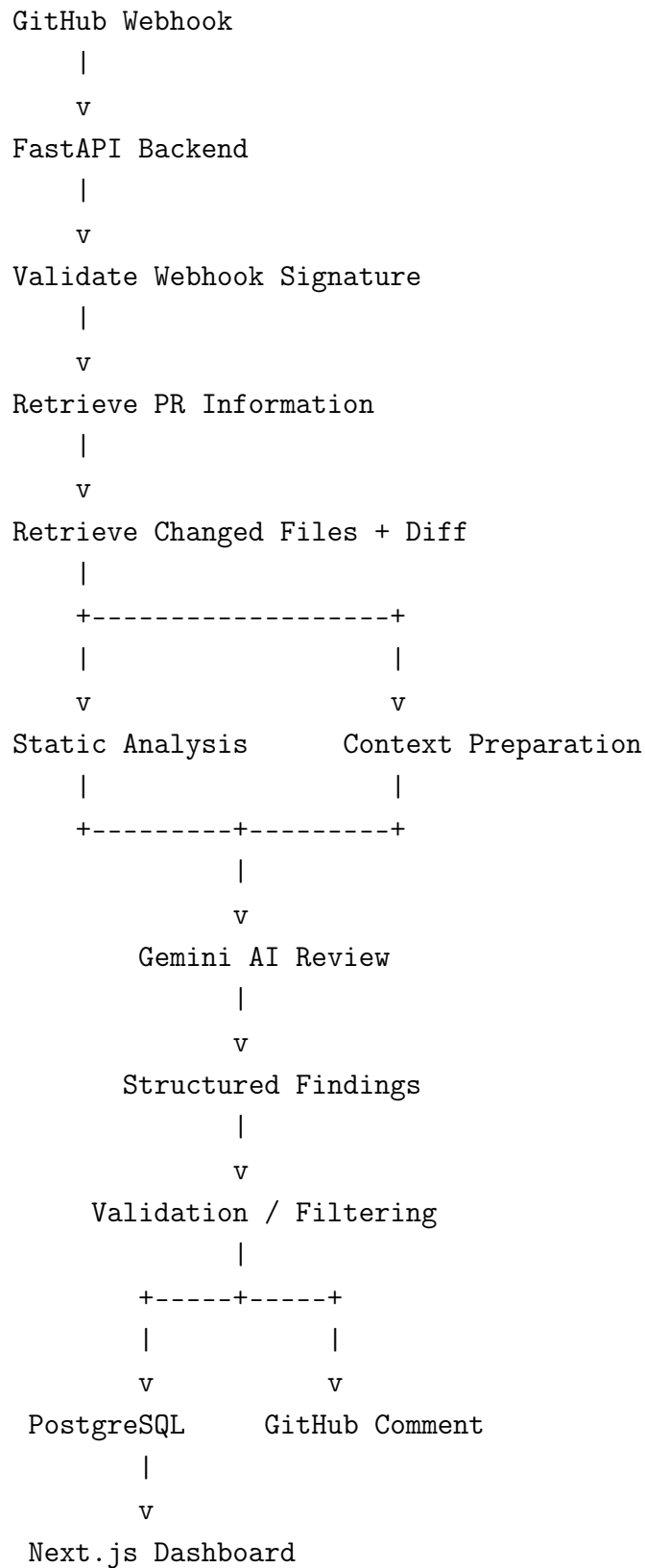
Potential users include:

- Individual software developers
- Junior developers
- Software engineering teams
- Open-source maintainers
- Technical leads
- Code reviewers
- Development teams using GitHub

6 Core System Workflow

The main workflow is:

```
Developer
  |
  v
Creates / Updates GitHub Pull Request
  |
  v
```



7 Technology Stack

Component	Technology	Purpose
Frontend	Next.js	Web dashboard
Frontend Language	TypeScript	Type-safe frontend development
UI	Tailwind CSS	Responsive interface design
Backend	FastAPI	REST API and webhook processing
Backend Language	Python	Backend and AI processing
Validation	Pydantic	Request and response validation
Database	PostgreSQL	Persistent application data
ORM	SQLAlchemy	Database interaction
AI	Gemini API	Contextual code analysis
Version Control	Git	Source control
Repository Platform	GitHub	Repository and Pull Request management
Integration	GitHub REST API	Repository and PR information
Events	GitHub Webhooks	Real-time Pull Request events
Static Analysis	Ruff / Bandit / ESLint	Deterministic code analysis
API Testing	Postman	Backend testing
Optional Vector Search	pgvector	Project-specific retrieval

8 Development Constraints

The project will be intentionally designed to run without Docker.

The local environment can use:

```
Next.js
|
npm run dev
|
localhost:3000
```

```
FastAPI
|
uvicorn main:app --reload
|
```

localhost:8000

PostgreSQL

|

Local PostgreSQL Service

AI

|

Gemini API

This avoids the need to run large local containers.

9 Major System Modules

9.1 GitHub Integration Module

Responsibilities:

- Connect repositories
- Receive GitHub webhook events
- Validate webhook signatures
- Retrieve Pull Request metadata
- Retrieve changed files
- Retrieve code diffs
- Post review comments

9.2 Webhook Processing Module

Example events:

- pull_request.opened
- pull_request.synchronize
- pull_request.reopened

Example endpoint:

POST /api/v1/webhooks/github

The backend should verify the GitHub webhook signature before processing the event.

9.3 Pull Request Analysis Module

This module extracts:

- Repository
- Pull Request number
- Author
- Base branch
- Head branch
- Changed files
- Added lines
- Removed lines
- Patch/diff information

9.4 Static Analysis Module

Before asking the LLM to analyse code, deterministic tools should be used whenever possible.

Possible tools include:

- Bandit for Python security analysis
- Ruff for Python linting
- ESLint for JavaScript and TypeScript

The static-analysis output can later be combined with AI analysis.

9.5 AI Review Engine

The AI Review Engine sends selected Pull Request context to Gemini.

The AI should not receive unnecessary repository information.

The review prompt should include:

- Changed code
- File name
- Programming language
- Relevant static-analysis findings
- Project rules where available
- Required structured response schema

10 AI Review Categories

The system can classify findings into categories such as:

- SECURITY
- BUG
- VALIDATION
- ERROR_HANDLING
- PERFORMANCE
- DATABASE
- MAINTAINABILITY
- CODE_QUALITY
- BEST_PRACTICE

11 Severity Classification

11.1 High Severity

Examples:

- SQL injection
- Authentication bypass
- Hard-coded sensitive credentials
- Critical data-loss possibility
- Major security vulnerability

11.2 Medium Severity

Examples:

- Missing validation
- Weak error handling
- Potential runtime failure
- Performance concern
- Unsafe assumptions

11.3 Low Severity

Examples:

- Naming problems
- Readability issues
- Small refactoring opportunities
- Maintainability improvements

12 Structured AI Output

The Gemini response should not be accepted as uncontrolled plain text.

The backend should request and validate structured output.

Example:

```
1 {  
2   "review_summary": {  
3     "risk_level": "HIGH",  
4     "total_findings": 2  
5   },  
6   "findings": [  
7     {  
8       "severity": "HIGH",  
9       "category": "SECURITY",  
10      "file": "app/routes/user.py",  
11      "line": 18,  
12      "title": "Potential SQL Injection",  
13      "problem": "User input is inserted directly into SQL.",  
14      "explanation": "An attacker may manipulate the query.",  
15      "suggestion": "Use parameterized queries.",  
16      "confidence": 0.94  
17    }  
18  ]  
19 }
```

Pydantic models should validate this response before it is stored or published.

13 Example Review Scenario

Suppose a developer submits:

```
1 @app.get("/user/{user_id}")  
2 def get_user(user_id):
```

```
3     query = f"SELECT * FROM users WHERE id = {user_id}"
4     return db.execute(query)
```

The system may produce:

Severity: HIGH

Category: SECURITY

Potential SQL Injection

The `user_id` value is directly inserted into the SQL query.
Untrusted input could alter the intended SQL statement.

Recommendation:

Use a parameterized query instead of string interpolation.

14 False Positive Reduction

A major goal of the project should be avoiding noisy reviews.

Possible strategies include:

1. Analyse changed code rather than the entire repository.
2. Use deterministic tools for problems they can reliably detect.
3. Ask the LLM for confidence information.
4. Apply confidence thresholds.
5. Avoid publishing extremely low-confidence findings.
6. Combine duplicate findings.
7. Allow developers to mark findings as useful or false positive.
8. Store feedback for later analysis.

15 Human-in-the-Loop Design

The system should not automatically claim that every detected issue is definitely a bug.

AI-generated findings are recommendations.

The developer or human reviewer retains final responsibility for deciding whether:

- The finding is valid
- The code should be changed

- The Pull Request should be approved
- The Pull Request should be rejected

Therefore:

AI assists the reviewer; AI does not replace the reviewer.

16 Database Design

Possible tables include:

- users
- repositories
- pull_requests
- reviews
- review_findings
- webhook_events
- project_rules
- developer_feedback

16.1 Users Table

Possible fields:

id
github_id
username
email
avatar_url
created_at
updated_at

16.2 Repositories Table

id
github_repository_id
owner
name
full_name
default_branch
is_active
created_at
updated_at

16.3 Pull Requests Table

id
repository_id
github_pr_number
title
author
base_branch
head_branch
status
commit_sha
created_at
updated_at

16.4 Reviews Table

id
pull_request_id
status
overall_risk
total_findings
analysis_started_at
analysis_completed_at
processing_time_ms
created_at

16.5 Review Findings Table

id
review_id
file_path

line_number
severity
category
title
problem
explanation
suggestion
confidence
source
created_at

16.6 Developer Feedback Table

id
finding_id
feedback_type
comment
created_at

Possible feedback types:

- USEFUL
- NOT_USEFUL
- FALSE_POSITIVE

17 Backend API Design

17.1 GitHub Webhook

POST /api/v1/webhooks/github

Receives GitHub events.

17.2 Repositories

GET /api/v1/repositories
GET /api/v1/repositories/{id}
POST /api/v1/repositories
DELETE /api/v1/repositories/{id}

17.3 Pull Requests

GET /api/v1/pull-requests
GET /api/v1/pull-requests/{id}

17.4 Reviews

GET /api/v1/reviews

GET /api/v1/reviews/{id}

POST /api/v1/reviews/{id}/rerun

17.5 Findings

GET /api/v1/reviews/{review_id}/findings

17.6 Feedback

POST /api/v1/findings/{finding_id}/feedback

18 Frontend Dashboard

The dashboard should initially remain simple.

18.1 Dashboard Home

Display:

- Connected repositories
- Pull Requests reviewed
- Total findings
- High-severity findings
- Recent reviews

18.2 Repositories Page

Display:

- Repository name
- Status
- Number of reviewed PRs
- Last review

18.3 Pull Requests Page

Example:

PR #42 - Add payment endpoint

Status: Reviewed

Risk: HIGH

Files Changed: 4

Findings: 5

HIGH 1

MEDIUM 3

LOW 1

18.4 Review Details

Each finding should display:

- File
- Line
- Severity
- Category
- Problem
- Explanation
- Suggested improvement
- Confidence

19 GitHub Review Output

The system should produce readable GitHub feedback.

Example:

AI Code Review

Overall Risk: HIGH

1 High | 2 Medium | 1 Low

HIGH - Security

File: app/routes/user.py

Line: 18

Potential SQL Injection

Why it matters:

User-controlled input is inserted directly into the SQL query.

Suggested improvement:

Use a parameterized query.

This review was generated automatically.

Please verify findings before making changes.

20 Project-Specific RAG

A later version can allow repositories to provide documents such as:

- coding-guidelines.md
- security-rules.md
- architecture.md
- api-standards.md
- CONTRIBUTING.md

These documents can be processed and converted into embeddings.

Relevant rules can then be retrieved for each review.

The workflow becomes:

Pull Request Diff

|

v

Determine Review Context

|

v

Retrieve Relevant Project Rules

|

v

Static Analysis Findings

|
v

Gemini

|
v

Context-Aware Review

This means the system can eventually review code according to the standards of the specific repository instead of relying only on generic software-engineering recommendations.

21 Security Requirements

Because the system handles repository information, security is important.

Requirements include:

1. Validate GitHub webhook signatures.
2. Never expose GitHub credentials in frontend code.
3. Store secrets using environment variables.
4. Avoid logging sensitive source code unnecessarily.
5. Request only required GitHub permissions.
6. Validate API inputs.
7. Protect dashboard endpoints.
8. Sanitize externally supplied data.
9. Avoid sending unnecessary repository content to the AI model.
10. Prevent unauthorized repository access.

Example environment variables:

```
DATABASE_URL=  
GITHUB_TOKEN=  
GITHUB_WEBHOOK_SECRET=  
GEMINI_API_KEY=
```

The `.env` file must never be committed to GitHub.

22 Recommended Project Structure

```
ai-code-review-assistant/  
|  
+-- backend/  
|  |  
|  +-- app/  
|  |  +-- api/  
|  |  +-- core/  
|  |  +-- models/  
|  |  +-- schemas/  
|  |  +-- services/  
|  |  |  +-- github/  
|  |  |  +-- ai/  
|  |  |  +-- analysis/  
|  |  |  +-- static_analysis/  
|  |  +-- repositories/  
|  |  +-- utils/  
|  |  +-- main.py  
|  |  
|  +-- tests/  
|  +-- requirements.txt  
|  +-- .env.example  
|  
+-- frontend/  
|  |  
|  +-- app/  
|  +-- components/  
|  +-- lib/  
|  +-- services/  
|  +-- types/  
|  +-- public/  
|  +-- package.json  
|  
+-- docs/  
|  +-- architecture/  
|  +-- api/  
|  +-- screenshots/  
|  
+-- README.md  
+-- .gitignore
```

23 Minimum Viable Product

The MVP should contain:

1. GitHub repository integration
2. Pull Request webhook detection
3. Webhook signature validation
4. Pull Request metadata retrieval
5. Changed-file retrieval
6. Diff extraction
7. Basic static analysis
8. Gemini-based code analysis
9. Structured findings
10. High/Medium/Low severity
11. PostgreSQL persistence
12. GitHub Pull Request review comment
13. Basic Next.js dashboard

Features such as RAG, advanced analytics and multi-repository administration should not delay the MVP.

24 Development Phases

24.1 Phase 1 – Foundation

Tasks:

- Create GitHub repository
- Create project folder structure
- Create FastAPI backend
- Create Next.js frontend
- Configure PostgreSQL
- Configure environment variables

- Create basic README

Expected Result:

Frontend, backend and database run successfully.

24.2 Phase 2 – GitHub Integration

Tasks:

- Configure GitHub integration
- Create webhook endpoint
- Validate webhook signature
- Detect Pull Request events
- Retrieve PR metadata
- Retrieve changed files
- Retrieve code diffs

Expected Result:

Creating a Pull Request causes the FastAPI application to receive the event and retrieve its changed files.

24.3 Phase 3 – Static Analysis

Tasks:

- Detect programming language
- Integrate Python analysis
- Integrate JavaScript/TypeScript analysis
- Normalize tool results

Expected Result:

The backend produces structured deterministic findings.

24.4 Phase 4 – Gemini Integration

Tasks:

- Connect Gemini API
- Create code-review prompt
- Send relevant diff context
- Request structured output
- Validate output using Pydantic
- Handle API errors
- Handle invalid responses

Expected Result:

A Pull Request diff produces structured AI review findings.

24.5 Phase 5 – Review Engine

Tasks:

- Combine static and AI findings
- Remove duplicates
- Apply severity rules
- Apply confidence threshold
- Generate review summary

Expected Result:

The system produces one consistent review result.

24.6 Phase 6 – GitHub Feedback

Tasks:

- Format review output
- Publish GitHub comment
- Avoid duplicate comments
- Update review when new commits are pushed

Expected Result:

The developer sees AI-generated feedback directly on the Pull Request.

24.7 Phase 7 – Dashboard

Tasks:

- Dashboard layout
- Repository page
- Pull Request page
- Review page
- Finding cards
- Severity filtering
- Review history

24.8 Phase 8 – Project Context / RAG

Tasks:

- Add project guideline documents
- Chunk documents
- Generate embeddings
- Store vectors
- Retrieve relevant standards
- Include retrieved rules in review context

24.9 Phase 9 – Evaluation

Tasks:

- Create test Pull Requests
- Introduce known issues
- Run automated reviews
- Record expected findings
- Compare expected and detected issues
- Measure false positives
- Measure review latency

25 Testing Strategy

Testing should include:

25.1 Unit Testing

Test:

- Webhook validation
- Diff parser
- Severity mapping
- AI response parser
- Finding deduplication

25.2 Integration Testing

Test:

- GitHub API integration
- Gemini integration
- PostgreSQL integration
- Webhook processing

25.3 End-to-End Testing

Example:

1. Create vulnerable code
2. Push branch
3. Create Pull Request
4. GitHub sends webhook
5. Backend analyses PR
6. Gemini generates findings
7. Finding appears on GitHub
8. Review appears in dashboard

26 Evaluation Metrics

The project should include measurable evaluation.

Possible metrics include:

26.1 Detection Rate

$$\text{Detection Rate} = \frac{\text{Correctly Detected Known Issues}}{\text{Total Known Issues}} \times 100$$

26.2 Precision

$$\text{Precision} = \frac{\text{True Positive Findings}}{\text{True Positive Findings} + \text{False Positive Findings}}$$

26.3 Recall

$$\text{Recall} = \frac{\text{True Positive Findings}}{\text{True Positive Findings} + \text{False Negative Findings}}$$

26.4 False Positive Rate

Measure how frequently the system reports a problem when the tested code does not contain the expected issue.

26.5 Severity Accuracy

Measure whether known test issues are assigned appropriate severity.

26.6 Review Latency

$$\text{Review Latency} = \text{Review Completion Time} - \text{Webhook Received Time}$$

Additional measurements may include:

- Files analysed
- Lines changed
- Number of findings
- AI processing time
- Static-analysis processing time

27 Evaluation Dataset

A controlled set of Pull Requests can be created containing known issues.

Examples:

- SQL injection
- Missing authorization
- Missing input validation
- Hard-coded secret
- Unsafe exception handling
- Potential null-value failure
- Inefficient database access
- Duplicate logic
- Poor naming
- Maintainability problems

Each test case should contain:

Test Case ID
Programming Language
Issue Type
Expected Severity
Expected File
Expected Line/Region
System Detection
Detected Severity
False Positive?
Review Time

28 Cost Strategy

The project is designed to minimize development costs.

Most technologies are open source or available without software licence fees:

- Next.js
- TypeScript
- Tailwind CSS

- Python
- FastAPI
- PostgreSQL
- SQLAlchemy
- Pydantic
- Git
- GitHub
- Ruff
- Bandit
- ESLint

Gemini API usage should be monitored according to the quota and billing conditions associated with the developer's Google account.

The architecture should avoid depending on paid infrastructure during initial development.

29 Possible Future Improvements

After completing the core system, future versions could include:

1. GitHub App installation flow
2. Multiple repositories
3. Organization support
4. Inline GitHub review comments
5. Custom repository rules
6. Advanced RAG
7. Repository architecture awareness
8. Historical review context
9. Team coding-standard enforcement
10. Developer feedback learning
11. Review analytics

12. CI/CD integration
13. Multiple AI model providers
14. Local LLM fallback
15. Review cost tracking
16. AI model comparison
17. IDE integration
18. Slack/Discord notifications
19. Automatic review summaries
20. Pull Request risk scoring
21. Security-focused review mode

30 Features That Should Not Be Added Initially

To prevent unnecessary scope expansion, the first version should avoid:

- Building a complete GitHub replacement
- Training a custom LLM
- Supporting every programming language
- Complex microservice architecture
- Kubernetes
- Docker dependency
- Enterprise billing
- Complex multi-tenant architecture
- Automatic Pull Request merging

These features are unnecessary for demonstrating the main engineering concept.

31 Portfolio Value

The project demonstrates several areas relevant to an Internship, Associate Software Engineer or Junior Software Engineer role.

31.1 Software Engineering

- Modular architecture
- API design
- Error handling
- Validation
- Testing
- Git workflows

31.2 Backend Engineering

- FastAPI
- REST APIs
- Webhooks
- PostgreSQL
- External API integration
- Asynchronous processing concepts

31.3 Frontend Engineering

- Next.js
- TypeScript
- Dashboard development
- API integration
- Responsive interfaces

31.4 AI Engineering

- LLM integration
- Prompt design
- Structured AI outputs
- Context management
- Confidence handling

- RAG
- AI evaluation

31.5 Security

- Webhook signature verification
- Secret management
- Repository permissions
- Input validation
- Secure API design

32 Potential Interview Discussion

The project can support technical interview discussions such as:

- Why were webhooks used instead of polling?
- How are GitHub webhook requests authenticated?
- How are Pull Request diffs retrieved?
- Why combine static analysis with an LLM?
- How are hallucinated AI findings handled?
- How are false positives measured?
- Why use structured output?
- How is the Gemini response validated?
- How would the architecture scale?
- How are repository credentials protected?
- Why use PostgreSQL?
- How could project-specific RAG improve review quality?
- How would you compare two different AI models?
- What happens if Gemini becomes unavailable?

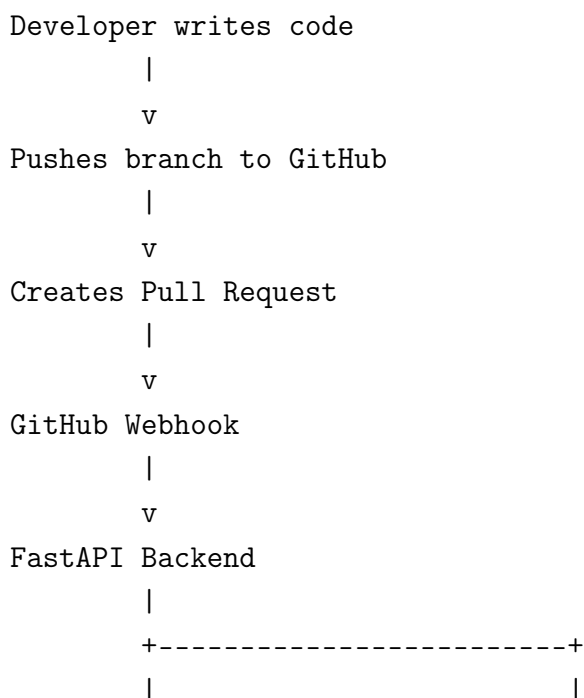
33 Success Criteria

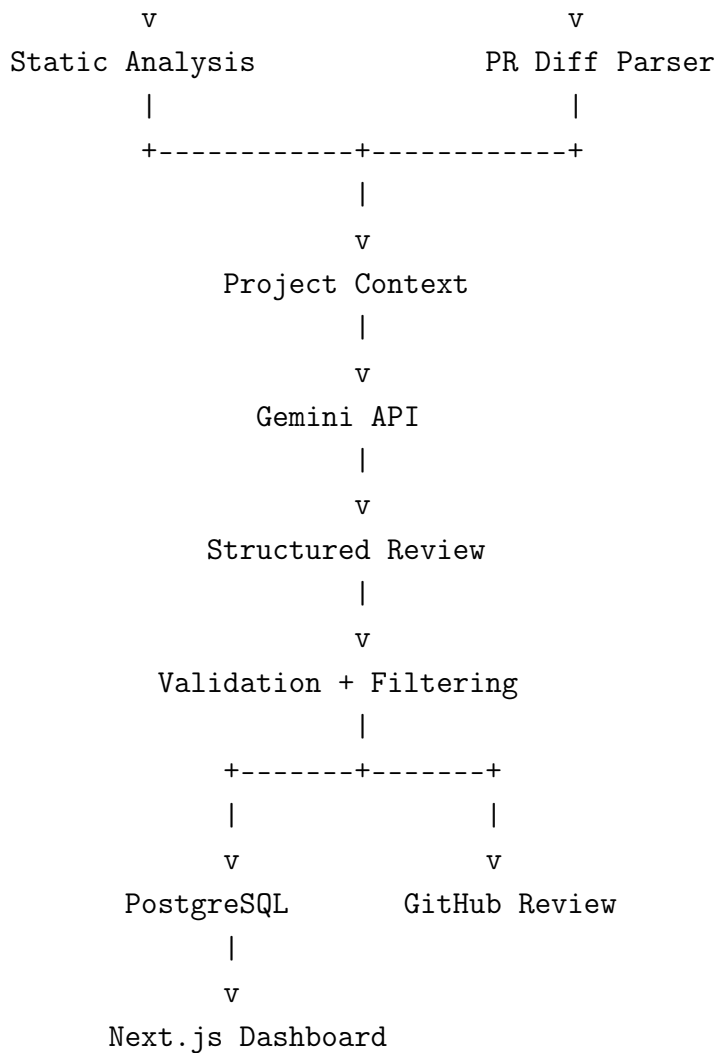
The MVP can be considered successful when:

1. A developer creates a GitHub Pull Request.
2. GitHub automatically sends an event to the backend.
3. The backend validates and processes the event.
4. Changed files and code diffs are retrieved.
5. Static analysis is executed where applicable.
6. Gemini analyses relevant changes.
7. Structured findings are successfully validated.
8. Findings are classified by severity.
9. Useful feedback is published to the GitHub Pull Request.
10. Review information is stored in PostgreSQL.
11. The review can be viewed through the Next.js dashboard.

34 Final System Vision

The final system should demonstrate the following complete workflow:





The key principle of the system is:

The purpose of AI is not to replace human code reviewers.

It provides an automated first-level review that helps developers identify potential problems earlier and gives human reviewers additional context for making better engineering decisions.

35 Recommended Development Principle

The project should be developed incrementally.

The first goal should **not** be to build the dashboard or advanced AI features.

The first major technical milestone should be:

GitHub Pull Request → Webhook → FastAPI → Retrieve Changed Files

Once this workflow operates reliably, Gemini analysis should be added.

After AI analysis works reliably, GitHub review publishing should be implemented.

Only after the complete backend workflow works should significant effort be spent on the dashboard.

Therefore, the recommended implementation order is:

**GitHub Integration → Analysis Engine → Gemini → GitHub Feedback →
Database → Dashboard → RAG → Evaluation**

36 Conclusion

The AI-Powered GitHub Code Review Assistant is designed as a practical software-engineering project that combines backend development, frontend development, database design, GitHub integration, security, static analysis and modern AI capabilities.

Unlike a simple chatbot application, the system participates in a real developer workflow by reacting to Pull Requests, analysing actual code changes and returning actionable feedback.

The project is intentionally structured so that a useful MVP can be developed locally without Docker or expensive infrastructure.

Most importantly, the project provides opportunities to demonstrate not only the ability to integrate an LLM, but also the ability to design, implement, test and evaluate a complete software system.